Deepfake Detection and Media Verification System

Project Overview, Requirements, Tech Stack and ML Model Mapping

1. What Is This Project?

The Deepfake Detection and Media Verification System is a full-stack web application that checks whether an uploaded image, video or audio file is authentic or has been manipulated or AI-generated. The user uploads a file, the system analyses it with several independent modules, combines their results into one confidence (suspicious) score, and produces a downloadable verification report.

Deepfakes are increasingly used for fraud, misinformation and identity abuse. This project gives users a single place to run forensic, metadata and machine-learning checks, and to see the evidence behind the result instead of only a yes or no answer.

1.1 Objectives

- Detect manipulated or AI-generated images, videos and audio.
- Extract text (OCR) and metadata from media and flag inconsistencies.
- Combine results from multiple modules into one confidence / suspicious score.
- Generate a clear verification report with evidence for each finding.
- Store submissions and results in a database for history and repeat lookups.

1.2 Core Modules

No.	Module	Purpose
1	Image verification	Detect face swaps, GAN or diffusion-generated images and edited regions.
2	Video verification	Analyse frames and temporal behaviour to detect deepfake videos.
3	Audio verification	Detect synthetic, cloned or spoofed voices.
4	OCR and metadata analysis	Read embedded text and inspect EXIF/XMP and file properties for tampering signs.
5	AI/ML based verification	Ensemble of pretrained and fine-tuned models behind a common interface.
6	Confidence / suspicious score	Fuse module outputs into a single score and label.
7	Report generation	Create a PDF report with scores, evidence, heatmaps and metadata.
8	Frontend and backend	Web UI for upload and results; API and processing pipeline.
9	Database integration	Store users, submissions, results and reports.

2. Requirements

2.1 Functional Requirements

- Users can upload image, video and audio files through the web interface.
- The system detects the file type and runs the matching analysis modules.
- Each module returns a score and supporting evidence.
- The system computes an overall suspicious score and a label: Likely Authentic, Suspicious or Likely Fake.
- Users can view results on a dashboard (scores, flagged frames, heatmaps, metadata table).
- Users can download a verification report as a PDF.
- Submissions and results are saved, and a repeated file (same hash) returns the stored result.

2.2 Non-Functional Requirements

- **Reliability:** Accuracy is reported honestly as a probability, with limitations stated in the UI and the report.
- **Performance:** Image analysis in seconds; video and audio run as background jobs with a progress indicator.
- **Security and privacy:** Validate file type and size, sanitise filenames, delete uploads after analysis, and never execute uploaded content.
- **Modularity:** Each module sits behind a common analyze(file) interface so models can be swapped without changing the app.
- **Usability:** Clean, simple interface that works on desktop and mobile browsers.

2.3 Hardware and Software Requirements

Item	Requirement
Development machine	Laptop with 8 GB RAM or more; GPU optional (CPU is enough for inference with small models).
Training environment	Free GPU from Kaggle or Google Colab for fine-tuning.
Operating system	Windows, Linux or macOS.
Language and runtime	Python 3.10 or newer.
External tools	ffmpeg, Tesseract OCR, exiftool.
Browser	Any modern browser (Chrome, Edge, Firefox).

3. Technology Stack

The whole stack is open source and free to use.

Layer	Technology	Used for
Backend	Python, Flask, Flask blueprints	REST API, routing, module orchestration.
Background jobs	concurrent.futures / threading (Celery + Redis later)	Running slow video and audio analysis.
Frontend	HTML, CSS, JavaScript, Bootstrap or Tailwind (Jinja templates)	Upload page, results dashboard, report download.
Database	SQLite (development), PostgreSQL (optional), SQLAlchemy ORM	Users, submissions, results, reports.

Layer	Technology	Used for
Image and video processing	OpenCV, Pillow, NumPy, ffmpeg	Frame extraction, ELA, preprocessing.
Deep learning	PyTorch, torchvision, Hugging Face Transformers	Loading and fine-tuning models.
Audio processing	librosa, torchaudio, ffmpeg	Spectrograms, MFCC features, audio extraction.
OCR	Tesseract or EasyOCR	Text extraction from images and video frames.
Metadata	exiftool, Pillow EXIF	EXIF/XMP and file property inspection.
Report generation	ReportLab or WeasyPrint	PDF verification reports.
Training and hosting	Kaggle / Colab (training), Hugging Face Spaces (free demo hosting)	Free model training and deployment.

4. ML Models and Which Function Each One Serves

Pretrained models are reused wherever possible and fine-tuned on small public dataset subsets. The application logic, scoring and reporting are built in this project.

Function	Model / technique	What it does	Data / source
Face detection (image and video)	MTCNN or MediaPipe Face Detection	Locates and crops faces before classification.	Pretrained, open source
Image deepfake classification	EfficientNet-B0 or Xception (fine-tuned); optional Vision Transformer from Hugging Face	Classifies a face or image as real or fake.	FaceForensics++, Celeb-DF subsets
Image manipulation forensics	Error Level Analysis (ELA) and noise residual analysis	Highlights edited or recompressed regions as a heatmap.	Classical image processing
Video frame analysis	Same image classifier applied to sampled frames (about 1 frame/sec) with mean and max aggregation	Scores each frame, then combines into a video score.	Frames from uploaded video
Video temporal checks	Blink-rate, lip-sync and frame-to-frame flicker checks (optional small LSTM)	Catches inconsistencies across time that single frames miss.	MediaPipe landmarks, DFDC subsets
Audio deepfake detection	Wav2Vec2 or AASIST; or CNN on mel-spectrograms	Detects synthetic or cloned speech.	ASVspoof, WaveFake subsets
Audio feature extraction	MFCC and mel-spectrogram (librosa)	Prepares audio input for the classifier.	Uploaded audio or video track
OCR	Tesseract or EasyOCR	Extracts text from images and frames to compare against metadata and context.	Pretrained, open source

Function	Model / technique	What it does	Data / source
Metadata analysis	Rule-based checks using exiftool / EXIF	Flags missing camera data, editing software tags, timestamp conflicts.	File metadata
Score fusion	Weighted average first; logistic regression later	Combines module scores into one suspicious score and confidence.	Validation results

4.1 Suspicious Score and Labels

Each module returns a score from 0 to 1 and its evidence. The overall score is a weighted combination, for example ML models 50%, forensics 25%, metadata 15% and audio 10% (weights are adjusted when a file has no audio or no metadata). The score maps to Likely Authentic, Suspicious or Likely Fake, and the confidence level is reduced when modules disagree.

5. Workflow

- User uploads a file through the web interface.
- Backend validates the file, computes its SHA-256 hash and checks the database for a previous result.
- The file type is detected and the matching modules run (image, video, audio, OCR and metadata).
- Module outputs are fused into the overall suspicious score and label.
- Results are saved to the database and shown on the dashboard.
- The user downloads the PDF verification report; the original upload is deleted.

6. Limitations and Responsible Use

- Detectors can fail on new generation methods they were not trained on, so results are probabilities, not proof.
- Scores can be affected by heavy compression, low resolution and noisy audio.
- Some datasets and models are licensed for research use only; licenses should be listed in the project README.
- All models, datasets and repositories used must be credited in the project documentation.